

Parity Blindness, Phase 3, and the **repS** Concern

How a residual faithfulness defect in the symcoder encoder was diagnosed and closed by switching from **repS** to **rep**

Echinocoder / symcoder working notes

2026-05-25 (initial), 2026-05-26 (resolution recorded)

Abstract

This document captures an investigation into a class of faithfulness defects in the symcoder encoder. Phase 1 + Phase 2 of the existing encoder were found to be parity-blind on 31 specific “shadow” events. A new “Phase 3” was added in which the deterministically faithful simplicial-complex multiset embedder is applied to the alignment table T of the event. In its first implementation Phase 3 closed 29 of the 31 cases. The two remaining shadows turned out to be a symptom of the more interesting structural problem the title alludes to: the alignment table was being built over **repS**, the pre-pruned “short” atom set, whose pruning had been justified *only* under the (now-known-false) assumption that pair-row marginals are sufficient. Switching Phase 3 to use the unpruned **rep** closed all 31 cases. The document records the chain of reasoning, the implementation, and the residual open questions, so that the work can be picked up cold from the repository.

Contents

1	What problem are we solving?	1
1.1	The encoder’s contract	1
1.2	The defect: parity blindness on structured events	1
1.3	Why Phase 1 + Phase 2 alone is insufficient	2
2	The attempted fix: Phase 3 simplicial-complex embedding	2
2.1	Architecture	2
2.2	First-pass result: 29 out of 31 shadow events fixed	2
3	The two intermediate failures: shadows 16 and 24	3
3.1	What the diagnostic shows	3
3.2	Why this is mathematically possible	3
3.3	The geometric cause	3
4	The repS concern (the key insight at the end of session)	3
4.1	Historical pruning	3
4.2	Why the pruning is now suspect	4
4.3	Concrete construction of rep	4
4.4	The hypothesis (now confirmed)	4
5	The resolution: switching Phase 3 from repS to rep	4
5.1	What was changed in the code	4
5.2	What this costs	5
5.3	Test status after the change	5

6	File locations and how to reproduce	5
6.1	Code under investigation	5
6.2	Tests	5
6.3	Experimental discovery script	6
6.4	One-shot diagnostic	6
7	Status and remaining open questions	6
7.1	Status today (after resolution)	6
7.2	Recommended follow-ups (no longer urgent, but worth doing)	6
7.3	Open theoretical question	7

1 What problem are we solving?

1.1 The encoder’s contract

Echinocoder’s symcoder encodes a labelled event E (a tuple of vectors in $\mathbb{R}^d$, labelled by particle names) as a fixed-length real vector that is invariant under:

- the *label-permutation group* $H = S_{n_1} \times S_{n_2} \times \cdots$ acting on particle labels of each type;
- the rotation group $SO(d)$ acting on the underlying vectors;
- (for $d = 3$ specifically) *not* the full $O(3)$: parity (reflection) is supposed to be preserved by the encoder because the operator set includes the pseudoscalar ε_3 .

End users use only the encoder; decoders exist solely as validation tools that confirm round-trips lose nothing other than the intended $H \times SO(d)$ ambiguity.

1.2 The defect: parity blindness on structured events

For “generic” real-valued events the existing two-phase encoder (Phase 1: per-orbit value multisets; Phase 2: per-row-pair joint value multisets) is empirically parity-faithful: $\text{encode}(E)$ differs from $\text{encode}(P(E))$ whenever E and $P(E) = \{-v : v \in E\}$ lie in distinct $SO(3) \times H$ orbits.

However, for certain *structured* events — typically with antipodal muons (e.g. $q = -p$) and integer-valued electron coordinates — the encoder produces *exactly the same output* for E and $P(E)$, despite the two being in different $SO(3) \times H$ orbits. This was discovered by the empirical search in `symcoder/experiments/encoding_fidelity_search.py` and pinned by 31 regression test cases in `symcoder/tests/test_parity_blindness_shadows.py`, parametrised over the fixture `symcoder/tests/_parity_shadow_events.py`.

The 31 “shadow events” are events for which $\text{encode}(E) = \text{encode}(P(E))$ exactly, yet the Gram-matrix plus signed-determinant invariant (Weyl’s First Fundamental Theorem for $SO(n)$) distinguishes them. All 31 are chiral in the strict sense: the chirality precondition in the test file confirms this case-by-case.

1.3 Why Phase 1 + Phase 2 alone is insufficient

For any antisymmetric operation (such as ε_3), the multiset of orbit values is intrinsically palindromic under negation (every value v appears together with $-v$ in the orbit). Hence Phase 1 *never* distinguishes parity for these rows.

Phase 2 (per-row-pair joint value multisets) usually breaks this palindrome on generic inputs but does not always: for the 31 shadow events the joint multisets are palindromic in the antisymmetric coordinate too.

This is a property of *pair-row* marginals. The information needed to distinguish chirality lives at higher arity (joint structure across ≥ 3 rows simultaneously); pair marginals miss it on structured inputs.

2 The attempted fix: Phase 3 simplicial-complex embedding

2.1 Architecture

A new encoder factory has been wired into `symcoder/encode.py`:

```
encode(plan, event,
        orbit_factory=..., # Phase 1
        phase2_factory=..., # Phase 2
        phase3_factory=Phase3SimplicialEncoderFactory() # default on
)
```

The new factory builds a `Phase3SimplicialEncoder` (defined in `symcoder/encoders/phase3_simplicial`), which:

1. Computes the full *alignment table* $T \in \mathbb{R}^{|G| \times |\text{repS}|}$ by direct evaluation: $T_{g,i} = \text{eval}(g \cdot \text{atom}_i, E)$ where the rows are indexed by all group elements $g \in G$ and the columns by all canonical `repS` atoms.
2. Treats T as a *multiset of $|G|$ tuples of dimension $|\text{repS}|$* (i.e. the column ordering is forgotten).
3. Feeds this multiset to the existing top-level `C0HomDeg1_simplicialComplex_embedder_2` simplicial embedder, which produces a deterministically faithful embedding of any multiset of n k -tuples in $2nk + 1 - k$ reals.
4. Concatenates that vector to the end of the existing Phase 1 + Phase 2 output, with a new `Phase3Tree` segment-descriptor entry so users can slice it off if they only want the smooth Phase 1+2 prefix.

2.2 First-pass result: 29 out of 31 shadow events fixed

With `phase3_factory` on by default and the alignment table built over `repS`, 29 of the 31 parity regression tests pass: `encode(E) ≠ encode(P(E))` at the required tolerance. A companion test (`test_parity_flip_no_phase3_still_blind`) explicitly passes `phase3_factory=None` and pins all 31 of the original failures as `xfail` so that the historical Phase-1+2-only bug remains visible.

The two surviving failures (shadows 16 and 24) led to the deeper finding described in §4.

3 The two intermediate failures: shadows 16 and 24

Note (post-resolution): the failures discussed in this section were observed when Phase 3 used `repS`. After switching Phase 3 to use the unpruned `rep` (see §6), both shadows now pass. The diagnostic content below is retained because it explains *why* the `repS` pruning was wrong — which is also the reason `repS` cannot in general be trusted as the column set of the alignment table.

3.1 What the diagnostic shows

For these two shadow events, the alignment tables $T(E)$ and $T(P(E))$ are *the same multiset of 12 column vectors*, bit-for-bit identical (zero difference at IEEE precision after lex sorting). Hence *any* encoder whose input is the multiset of columns of T must produce identical output for these two events.

This is not a numerical artefact of the simplicial encoder’s piecewise-linear corner structure. It was checked with `np.array_equal` on lex-sorted rows.

3.2 Why this is mathematically possible

The label group H acts on T (viewed as a function $G \rightarrow \mathbb{R}^{|\text{repS}|}$) by *left translation*: $T(\sigma \cdot E)(g) = T(E)(\sigma^{-1} \cdot g)$. An H -invariant encoding need only be invariant under this left-translation action.

The multiset-of-columns is invariant under *any* permutation of columns — including permutations that are *not* left translations by any $\sigma \in H$. So “multiset of columns” is a *strictly coarser* equivalence than “function modulo left translation by H ”.

In other words: two distinct functions $T_1, T_2 : G \rightarrow \mathbb{R}^{|\text{repS}|}$ can give the same column-multiset without being in the same H -orbit. For shadows 16 and 24 this happens; the encoder consequently cannot distinguish them.

3.3 The geometric cause

For these events $T(E)$ has a high-symmetry column pattern: many columns share the same Phase-1-style (mag, dot) profile, and within each such cluster the ε_3 values come in $\pm$ pairs. A column-permutation that pairs each $+$ -eps column with its $-$ -eps counterpart, while leaving the (mag, dot) profile per cluster unchanged, takes $T(E)$ to $T(P(E))$ as a multiset — but this permutation is *not* of the form “left translation by some $\sigma \in H$.”

4 The repS concern (the key insight at the end of session)

4.1 Historical pruning

The **repS** set used in the current code is the result of a historical pruning process:

- **rep** (original): *all* mags, all (dissimilar) dot pairs, and all (dissimilar) ε_3 triples across the labels of the plan. By Weyl’s First Fundamental Theorem this is information-complete for $SO(3)$ invariants of the labelled vector tuple.
- **repL** (intermediate): a longer-than-**repS** intermediate pruning, no longer present in the codebase.
- **repS** (current, “short”): further pruned by observing that many pairs of **rep** rows are G -orbit-equivalent (e.g. $(\text{dot}(a, b), \text{dot}(c, d))$ and $(\text{dot}(a, d), \text{dot}(b, c))$ both have the same G -orbit at the pair level), so only one of each duplicate pair needs to be stored.

4.2 Why the pruning is now suspect

The pruning from **rep** $\rightarrow$ **repL** $\rightarrow$ **repS** was justified *under the assumption that Phase 1 + Phase 2 multisets are sufficient*: the duplicates being eliminated were duplicates *at the pair-row marginal level*.

We now know (from the 31 shadow events) that pair-row marginals are *not* sufficient for faithful encoding. Therefore the pruning that yielded **repS** was based on a false premise: rows that look “duplicate” at the pair-marginal level may carry distinct joint information once we look at higher-arity marginals (such as the full alignment table).

In particular, feeding **repS**-derived T to the simplicial encoder makes the same mistake the pair-only pruning made: it throws away rows whose presence might break the column-multiset palindrome.

4.3 Concrete construction of rep

The full **rep** can be regenerated from a plan as the concatenation of:

1. $[\text{mag}(v) : v \in \text{labels}]$ — one mag per label.
2. $[\text{dot}(a, b) : \{a, b\} \subset \text{labels}, a \neq b]$ — one dot per *unordered* pair of distinct labels. Iterating over *combinations* (not permutations) avoids double-counting; the dot constructor already canonicalises its arguments.
3. $[\varepsilon_3(a, b, c) : \{a, b, c\} \subset \text{labels}, a, b, c \text{ distinct}]$ — one ε_3 per *unordered* triple of distinct labels. Again iteration over combinations is enough; the ε_3 constructor is antisymmetric and canonicalises.

For a 3-electron, 2-muon plan:

$$|\text{rep}| = \underbrace{5}_{\text{mags}} + \underbrace{\binom{5}{2}}_{\text{dots}=10} + \underbrace{\binom{5}{3}}_{\varepsilon_3=10} = 25,$$

versus $|\text{repS}| = 8$ currently used. T becomes $|G| \times |\text{rep}| = 12 \times 25 = 300$ entries. The simplicial embedder size then becomes $2 \cdot 12 \cdot 25 + 1 - 25 = 576$ reals, replacing the current 185.

4.4 The hypothesis (now confirmed)

Using `rep` (not `repS`) in the Phase 3 alignment table *should* make the column-multiset of T injective on $SO(3) \times H$ orbits for the two stubborn shadow events. When this swap was implemented, all 31 shadow events — including 16 and 24 — did indeed become distinguishable by the encoder. See §6 for the implementation and the resulting test status.

An important caveat that has not been disproven. Even with `rep`, the column-multiset of T is a strictly coarser equivalence than “function modulo left-translation by H ”. We have no theorem that says `rep`’s column-multiset is a complete invariant for $SO(3) \times H$ in general. All we know is that on the 31 events of the current fixture it suffices; an adversarial event constructed against the column-multiset of `rep` is still mathematically possible (just harder to find than against `repS`). See §7.2.

5 The resolution: switching Phase 3 from `repS` to `rep`

5.1 What was changed in the code

1. A new function `rep(context, operations)` was added to `symatom/rep.py`, exported from `symatom/__init__.py`. It flattens `repS`’s `FlavouredOperators` into a list of individual `Atoms`: every atom across every G -orbit, with one atom per sign-canonical form. For symmetric ops this is one atom per unordered label combination of the operation’s rank; for antisymmetric ops it is one atom per unordered label combination (the $\pm$ pair is collapsed by storing only the `sign=+1` representative).
2. `symcoder/encoders/phase3.simplicial.py` was switched from `repS` to `rep`. The alignment table is now $(|G|, |\text{rep}|)$ instead of $(|G|, |\text{repS}|)$.
3. No other production code was touched. Phase 1 and Phase 2 continue to use `repS` internally, which is correct: the `repS` pruning was justified at the pair-marginal level, and Phase 1 + Phase 2 only consume pair marginals. Only Phase 3, which claims full faithfulness, needed `rep`.

5.2 What this costs

For the 3-electron, 2-muon parity-fixture plan:

	<code>repS</code>	<code>rep</code>
# atoms ($= k$)	8	25
alignment table shape	12×8	12×25
Phase 3 output size $2nk + 1 - k$	185 reals	576 reals
total encoder output (Ph1+Ph2+Ph3)	≈ 800 reals	≈ 1200 reals

The growth scales as roughly $\binom{|\text{labels}|}{\text{rank of } \varepsilon}$ for the ε row count, which is the dominant cost for plans with many particles and rank-3 (or higher) antisymmetric operations. This is a known and accepted cost; it is the price of taking the unpruned information-complete set of rows.

5.3 Test status after the change

- `test_parity_flip_distinguishable` (Phase 3 on, `rep`-based): **31 of 31 pass**.
- `test_parity_flip_no_phase3_still_blind` (Phase 3 off, pinning the historical bug): 31 of 31 xfail (as expected).
- Total `symcoder/symatom` suite: 622 passed, 32 xfailed. The 32 xfails are 1 original alignment-decoder xfail + the 31 historical-no-Phase-3 xfails.

6 File locations and how to reproduce

6.1 Code under investigation

- `symcoder/encode.py` — top-level encode functions; Phase 3 wired in with default-on `phase3_factory` sentinel.
- `symcoder/encoders/phase3_simplicial.py` — the new `Phase3SimplicialEncoder` + factory. Now uses the unpruned `rep`.
- `symatom/rep.py` — defines both `repS` (the historical pruned set, still used by Phase 1 + Phase 2) and the new `rep` function used by Phase 3.
- `symcoder/describe.py` — `SegmentInfo`, `Phase3Tree`, `EncodingTree.phase3`.
- `COHomDeg1_simplicialComplex_embedder_2_for_array_of_reals_as_multiset.py` — the underlying simplicial-complex multiset embedder (top-level Echinocoder file, not part of any package). Output size $2nk + 1 - k$ for input shape (n, k) .

6.2 Tests

- `symcoder/tests/test_parity_blindness_shadows.py`:
 - `test_parity_flip_distinguishable` (Phase 3 on): 31/31 pass after the `repS` → `rep` fix.
 - `test_parity_flip_no_phase3_still_blind` (Phase 3 off, xfail-pinned): 31/31 xfail.
- `symcoder/tests/_parity_shadow_events.py` — auto-generated fixture; 31 shadow events as Python literals.

6.3 Experimental discovery script

`symcoder/experiments/encoding_fidelity_search.py` — brute-force search over event families; $SO(3)$ and $O(3)$ oracles based on Gram-plus-signed-dets up to typed permutations of labels. Re-runnable to discover further shadow events or to validate that a fix really fixes *all* known shadows.

6.4 One-shot diagnostic

To check whether $T(E)$ and $T(P(E))$ are the same multiset for any event, see the snippet near the end of the chat: build the table with `Phase3SimplicialEncoder._build_table`, lex-sort the rows, compare with `np.array_equal`.

7 Status and remaining open questions

7.1 Status today (after resolution)

- Phase 3 is implemented, wired in, default-on, and uses the unpruned `rep` as the alignment-table column set.

- Test suite: **622 passed, 32 xfailed**. The 32 xfails are 1 pre-existing alignment-decoder xfail + 31 historical-no-Phase-3 xfails. All 31 parity-blindness regression cases pass with Phase 3 on.
- Both demos (`demo_roundtrip.py`, `demo_shadow_event.py`) still build; the parity-blindness demo PDFs are visibly distinguishable now that Phase 3 is on.
- No production code outside Phase 3 was changed. Phase 1 and Phase 2 continue to use `repS`; that is correct because the `repS` pruning is valid at the pair-marginal level.

7.2 Recommended follow-ups (no longer urgent, but worth doing)

1. **Stress-test the `rep` column-multiset faithfulness.** The current fixture has 31 shadow events; `rep` clears all 31, but does not prove the column-multiset of $T(\text{rep})$ is a complete $SO(3) \times H$ invariant in general. Extending `encoding_fidelity_search.py` to search against the `rep`-based encoder (rather than the `repS` one) might surface fresh adversarial events.
2. **Document the dual role of `repS` vs `rep`.** The current docstrings explain why Phase 3 uses `rep`, but the broader narrative “`repS` for pair-marginal encoders only; `rep` for whole-table encoders that claim faithfulness” should probably be surfaced in `SPEC.md` as a top-level design principle.
3. **Per-row sparsity / sub-selection of `rep`.** `rep` grows as $\binom{|\text{labels}|}{\text{rank}}$ which is painful for large plans. Worth investigating whether there is a structured *subset* of `rep` (call it `repM` for medium) which is still sufficient for whole-table faithfulness but smaller. Any such subset must be characterised by a proof, not a heuristic, since the history of `repS` is a cautionary tale.

7.3 Open theoretical question

- Multiset-of-columns of $T(\text{rep})$ is still a strictly coarser equivalence than “function modulo left-translation by H ”. Even with `rep` fixed, palindromic column-multisets may remain possible in principle.
- If so, the next architectural lever is to encode T as a function modulo H (e.g. via canonical lex-min over the $|H|$ left-translations of T) rather than as a multiset. This is more corner-prone than simplicial but is the only obvious way to recover the lost group-labelling structure.
- Alternatively, augment T with derived rows (e.g. row-products of antisymmetric and symmetric rows) whose presence breaks the column-multiset palindrome. Construction of provably non-palindromic augmentations is an open problem.

Appendix: explicit `rep` construction recipe

Note: this recipe was the design proposal during the investigation. The actual implementation in `symatom/rep.py` achieves the same result by reusing existing machinery: it constructs a `FlavouredOperator` for every (operation, flavour) pair, then iterates over each `FlavouredOperator.atoms_one`. Whether you read the recipe as “iterate combinations across all labels for each operation rank” (this appendix) or “flatten every G-orbit in `repS`” (the implementation), the resulting atom list is identical.

Given a `Plan` with `context.types[i].labels` for each particle type i , and any choice of operations (rank-1 symmetric, rank-2 symmetric, rank-3 antisymmetric, etc.):

1. Let `labels` be the concatenated tuple of all labels across all types.

2. Emit one $\text{mag}(\ell)$ atom for each $\ell \in \text{labels}$.
3. Emit one $\text{dot}(\ell_i, \ell_j)$ atom for each unordered pair $\{\ell_i, \ell_j\} \subseteq \text{labels}$ with $\ell_i \neq \ell_j$ (i.e. `itertools.combinations(labels, 2)`).
4. Emit one $\varepsilon_3(\ell_i, \ell_j, \ell_k)$ atom for each unordered triple $\{\ell_i, \ell_j, \ell_k\} \subseteq \text{labels}$ with the three labels pairwise distinct (i.e. `itertools.combinations(labels, 3)`).

The `dot` and ε_3 atom constructors are responsible for their own canonical-form sorting (lex order for symmetric ops, sign extraction for antisymmetric ops), so iterating over `combinations(unordered)` is sufficient.

For multi-type plans the labels of different types are concatenated; no special treatment is required because `mag`, `dot`, and ε_3 operate on labels independent of which type each label belongs to.